VMM PO Governance — Developer Handover (code v1.1.4)

- **For:** the software engineer building the SAP R/3 and BW integration
- **From:** Planning (tool owner: kosaraju.srihari@vishalwholesale.co.in)
- **Date:** 13 Oct 2026
- **Read with:** *Developer Brief (Stage 1 BW · Stage 2 SAP R/3 · Stage 3 AI)*, *SAP API Specification v1.4* + `po-governance-sap-api.yaml`, *Functional Logic Guide* — all in the `docs/` folder of this package (section 0).

> **Important:** some developments are pending (sections 0.1 and 8). The API contract in Specification v1.4 is the agreed direction, but some fields and verdict values **will change**. Please start with the items marked "safe to start now" (section 9). Anything marked "wait" should not be built in final form until we confirm it.

0. The developer package at a glance

#	Document / file	What it gives you
1	*Developer Brief — Stage 1 BW · Stage 2 SAP R/3 · Stage 3 AI*	Target design: data flows, timings, SAP touch points
2	*Functional Logic Guide*	Every decision rule in plain English
3	*SAP API Specification v1.4* (PDF) + `po-governance-sap-api.yaml` (OpenAPI 3.0)	Interface contract SAP ↔ tool: /ping, /events, /decisions, /reason-codes
4	**This handover**	Current stage, where things are in the code, pending developments
5	**Source code v1.1.4**	Everything running today on .35 and .37

0.1 Current stage and pending developments

Area	Current stage	Pending development
Decision rules (single concept, multi concept true repeat / BOBs, new article)	Built and in use; run **in the browser**	Move to the **server** (needed for `/decisions`)
Data	Daily **manual uploads** of BW / SAP extracts (stock, pending PO, sales, plan, BSR, masters)	**BW 04:00 feed** loader; **SAP live** PO / GRC / GRT events
Users, roles, MC scope	Built: logins from SCR file, own-MC restriction, admin screens	—
Watch List	Built **inside the tool**: category head approves / rejects with comment; buyer raises PO in SAP manually	Watch List as **SAP workflow** step with reason codes
Same-day control	In-tool ledger of today's approved single concept quantities	Replaced by **live open-PO position** from SAP events
Recommendations	Single concept: colours × months with Max Permissible and today's approved. Multi concept: attribute + colour ranking with an indicative quantity	**Multi concept recommended quantity per option (P1)**
Released POs	Tool 2 (planner batch check) only	**Action on released POs** workflow + day-end report
SAP interface	Not built; specification v1.4 agreed	`/api/sap/v1/ping, events, decisions, reason-codes`, HTTPS 8443, technical users, IP allow-list, shadow mode, monitoring
Storage	PostgreSQL on .37 (files on .35)	Live-position tables, decision log (schema v3)

Area	Current stage	Pending development
AI layer (Stage 3)	Not built	After Stages 1–2 run steadily

1. Where things stand

Server	Address	Role	Storage	Version
.35	http://10.10.16.35:8080	Category teams' hands-on (live users). **Do not develop or test here.**	JSON files	v1.1.4
.37	http://10.10.16.37:8080	**Build server**: all development, SAP/BW integration testing, later production for everyone	PostgreSQL 17 (localhost:5433)	v1.1.4, then v2.x

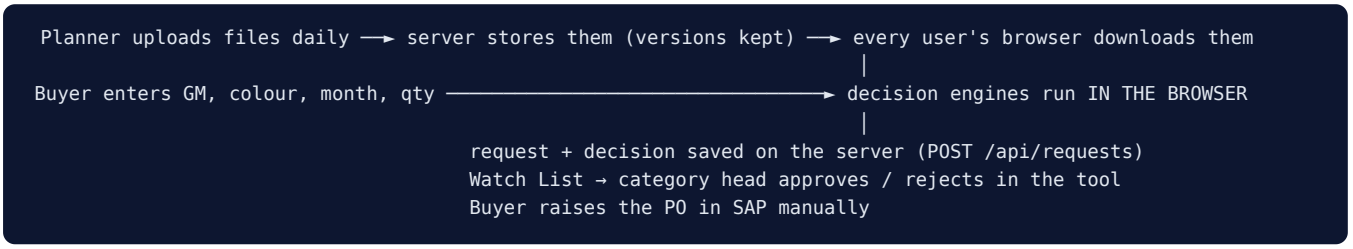
Both servers run the **same code (v1.1.4)**. New development goes to **.37 only** as v2.x. .35 receives bug fixes only.

2. What is in this package

Path	What it is
`server.py`	Web server (Python 3.9+, standard library only). Serves the page and all `/api/*` endpoints.
`accounts.py`	Logins, roles, sessions, SCR mapping (which buyer / category head owns which Brand × MC × MRP).
`storage.py`	Storage layer: `FileStore` (JSON files, .35) and `PgStore` (PostgreSQL, .37), same methods.
`setup_db.py`, `migrate_to_db.py`, `install-database.ps1`	Create the PostgreSQL database, copy data into it, switch over.
`pylib/`	Bundled pure-Python PostgreSQL driver (`pg8000`). No `pip install` needed.
`app/`	The browser side: `index.html` (the tool), `shared.js` (server sync), `users.js`, `history.js`, `feedback.js`, `login.html`.
`orig.html`, `build.py`	**Source of `app/index.html`.** `orig.html` is the business-approved v89 (6) tool; `build.py` applies ~88 exact-text patches and writes `app/index.html`.
`tests/`	End-to-end test scripts (Python + Playwright) used before every release.
`docs/`	Developer Brief, API Specification v1.4 (PDF + OpenAPI YAML), Functional Logic Guide.
`config.example.json`	All settings with explanations. The real `config.json` on the servers is **not** included (it holds the database password).

Not included: user data, the `pgsql` PostgreSQL binaries for Windows (separate zip), the how-to video.

3. How the system works today



- **The decision rules run in the browser today**, in `app/index.html`. The server stores data, requests, approvals, users and audit, but **does not compute decisions**. For SAP's `/decisions` call the rules must move to the server (section 8, item 1).
- **Same-day ledger:** approved quantities of today's single concept requests (all users) are added to pending POs so the same headroom cannot be used twice in a day. With SAP live, this is replaced by the live open-PO position from `/events`.

- **Sign-in:** session cookie + header `X-VMM: 1` on every non-GET call. Roles: admin, planner, cathead, buyer. Buyers and category heads can only validate their own Brand × MC × MRP (from the SCR products file).

Run it on your PC

```
python server.py                     # http://localhost:8080, files in .\data (creates admin on first start)
python server.py --check             # what is stored
python setup_db.py && python migrate_to_db.py   # optional: PostgreSQL mode (needs .\pgsql)
```

First sign-in: the `admin_email` in `config.json` with the `default_password` (1234), then set a new password. Upload the data files (Data Setup, or one zip) to see real results.

Making changes to the page

Never edit `app/index.html` directly. Change `build.py` (add a `rep("""old""", """new""", label=...)` patch in section 11) or the small `.js` files, then run `python build.py` from this folder (it fails loudly if a patch no longer matches). This keeps every change to the approved v89 rules visible and reviewable.

4. Where the rules are (`app/index.html`, v1.1.4 line numbers)

Function	Line	What it does
`resolveGM`	3740	GM → Brand, MC, MRP, Flag, Division (Universe Master / Article Master)
`routeRequest`	3825	Picks the engine: in Universe Master → **B (single concept)**; in BSR with this colour → **A True Repeat**; in BSR other colour → **A BOBs**; else **NEW**
`runEngineB`	3847	Single concept: colour mix gate (< 2% → Watch List), days-of-cover in the delivery month vs 75 d + 5 %, Max Permissible Qty
`quartile`, `pctRank`	4229, 4242	Rank PDSTR within the peer pool
`absBlock`	4257	Absolute block: PDSTR < 1 %/day with WOF ≥ 4 weeks → Block
`runEngineA`	4270	Multi concept: peer pool (same MC/MRP/Flag, WOF ≥ 4, current seasons, fallback if < 5), quartiles, 10 % border zone, BOBs sibling logic
`_confirmFinalQty`	1844	Pop-up to confirm a quantity up to Max Permissible
`_approvedQtyFor`	1882	Which approved quantity counts in the same-day ledger (single concept only)
`formSubmit`	8471	Validate button: runs routing + engine, saves the request
`_buildF2RawMap`, `_mergeSessionLedgerIntoF2Map`	10223, 10240	Pending POs per colour per month + today's approvals
`_runB`, `_runA`	10441, 10593	View Recommendation (single / multi concept)
`runEngineB2`	5090	Tool 2 (planner batch assessment of released POs)
`_strCaution`	9935	Information-only STR heads-up on Q1/Q2 Allow

Parameters (planner-editable, stored on the server): `T1 75` days, `strBuffer 5` %, `mix 2` %, `mixWindowDays 45`, `wof 4` weeks, `peer 5`, `borderTolerance 10` %, `D 75`, `N 2`, seasons. Full rules in plain English: *Functional Logic Guide* and the tool's "How it works" tab.

5. Input data

Key	File	Today	Future source (Developer Brief, Stage 1/2)
f1	F1 Stock (closing stock by GM × colour)	manual daily upload	BW 04:00 (previous day closing)
f2	F2 Pending PO (released, by delivery month)	manual daily	BW 04:00 + SAP live PO events

Key	File	Today	Future source (Developer Brief, Stage 1/2)
f3	F3 Sales trend (daily, 45 days)	manual daily	BW 04:00
f4	F4 Forecast / plan (Brand × MC × MRP × Flag × month)	manual	manual, later SAP
um	Universe Master (single concept GMs)	manual	BW
artmaster	Article Master (attributes, season)	manual	BW
bsr	BSR per division × season (WOF, per-day sell-through)	manual (several files, combined)	BW weekly
colmaster	Colour Master	manual	SAP
attrrel	Attribute Relevance (MC × MRP → attribute)	manual	manual
catheadmap	SCR products (Brand × MC × MRP → Level 1/2 buyer, category head emails)	manual	manual

Uploads are parsed in the browser and stored as gzipped JSON rows per version (`dataset_versions` table).

6. Database (PostgreSQL, `storage.py` → `PgStore`)

Table	Holds
`app_users`, `app_sessions`	Logins (PBKDF2 hashes), sessions
`app_settings`, `doc_stamps`	Parameters, manifest, SCR mapping, small documents
`dataset_versions`	Every uploaded data file version (gzipped JSON)
`upload_log`, `upload_batches`	Upload history (Upload IDs)
`request_events`	Tool 1 requests, approvals and "clear" events, per day (event log; replayed to get the current state)
`run_index`, `run_files`	Saved Tool 2 runs
`audit_log`, `feedback_log`	Audit trail, user feedback
`schema_info`	Schema version (currently 2)

The SAP live-position tables (open PO by item, goods movements, parked events, decision log) **do not exist yet**. They are part of the integration build (Spec v1.4 §5–6) and will be added as schema version 3.

7. APIs

7a. Existing internal API (browser ↔ server, session cookie) — *not for SAP*

`/api/status`, `/api/health`, `/api/auth/*`, `/api/users/*`, `/api/data/*`, `/api/uploads`, `/api/history/*`, `/api/runs/*`, `/api/params`, `/api/requests`, `/api/approvals/*`, `/api/feedback/*`, `/api/backups`, `/api/audit`, `/api/ai/*`. Handlers are `api_<METHOD>_<name>` in `server.py`. These may change between versions without notice.

7b. SAP-facing API to build (Spec v1.4, `/api/sap/v1/…`)

Endpoint	Caller	Purpose
`GET /ping`	SM59 / health job every minute	Connection test; 3 failures → SAP switches the governance step off
`POST /events`	bgRFC queue from BAdIs	`PO_CREATED`, `PO_CHANGED`, `PO_RELEASED`, `PO_RELEASE_REVOKED`, `PO_CLOSED`, `PO_DELETED`, `GR_POSTED`, `GR_REVERSED` (GRC), `RETURN_POSTED`, `RETURN_REVERSED` (GRT), `GOV_OUTCOME` — stored, de-duplicated, applied to open PO / stock

Endpoint	Caller	Purpose
`POST /decisions`	Release workflow step after OTB + depth, ≤ 3 s	Verdict per PO item with max qty, reason, route
`GET /reason-codes`	SAP approver screen	Exception reason list

Security: HTTPS on 8443 with the company certificate, Basic-auth technical users, IP allow-list (Spec §2). None of this exists yet on the tool side.

8. Pending developments that will cause changes

#	Development	What changes	Impact on your work	Status
1	**Decision engines moved from the browser to the server**	Rules re-implemented server-side, verified against browser results on real data	`/decisions` can only return real verdicts after this. Contract stays as Spec v1.4.	Next on .37 (tool side)
2	**Verdict mapping tool → SAP**	Tool results are *Auto Allow, Allow, Hold – Don't Raise, Block, Watch List*; the spec has *ALLOW / REDUCE / WATCH / HOLD*	Proposed: Auto Allow / Allow → ALLOW; quantity above Max Permissible → REDUCE + `max_qty`; Max Permissible 0 → HOLD; Watch List → WATCH (route CATHEAD); **Block → HOLD with reason or a new BLOCK value (breaking change)**.	**Decision pending — wait**
3	**Multi concept: recommended quantity per option**	Today multi concept results do not use quantity; a quantity rule per option (P1) will be added	May add `max_qty` / metrics for multi concept items in `/decisions`. Same-day counting of multi concept approvals added.	Rules pending from business
4	**Action on released POs** (nightly check of released POs vs decisions; buyer keep / reduce / cancel with reason; category head agree / reduce / cancel; day-end report)	New workflow and report	Defines the content of `GOV_OUTCOME` events and possibly new event types / reason codes; who executes the PO change in SAP is open.	Case list awaiting sign-off
5	**Reason codes, B&M head limit, SAP approval levels**	Final code list and escalation rule	`GET /reason-codes` content, `bm_head_required`, `route_to` values	Open with business
6	**GRT (goods return)**	Does a return re-open pending PO quantity?	How `RETURN_POSTED` / `RETURN_REVERSED` are applied to open PO quantity	Open — confirm with SAP team
7	**BW 04:00 extracts and cut-off**	Exact extract list, control file, cut-off time; POS posting time vs 04:00	Event cut-off rule (`occurred_at` after cut-off), double-counting prevention	Open — confirm with BW team
8	**Spec v1.5 items** (cut-off, released-PO rule, depth bypass only for tool-approved qty below minimum depth, BW as pending-PO source)	Spec update	Small field / rule changes	To be issued after items 2, 6, 7
9	**Same-day ledger switched off when SAP is live**	Live open-PO position replaces it	None for SAP; tool side only	With go-live
10	Category head approval rationale mandatory; optional one-time passwords instead of 1234	Tool screens	None for SAP	Optional

9. What to start now and what to wait for

Safe to start now (unlikely to change): * SM59 destination, technical user, HTTPS/certificate plan, IP allow-list with network team. * BAdIs `ME_PROCESS_PO_CUST` and `MB_DOCUMENT_BADI` → bgRFC ordered queue → `POST /events` payloads for PO and goods movements exactly as Spec v1.4 §5 (message id, sequence, retries until HTTP 200). * Release-workflow background step calling `POST /decisions` with the 3-second limit and the **NOT_CHECKED** fallback, the `/ping` health job and automatic switch-off (Spec §6, §10). * Develop against a **mock** generated from `po-governance-sap-api.yaml` (e.g. Postman mock or Prism) until the tool side endpoints are live on .37.

Wait for confirmation (items 2–8): final verdict values / `BLOCK`, routing values, multi concept quantity fields, `GOV_OUTCOME` content for the released-PO workflow, reason-code list, GRT handling, BW cut-off.

Tool-side endpoints (`/api/sap/v1/*` , live-position tables, decision log, monitoring page): ownership to be agreed. Recommended: Planning builds the tool side on .37 together with item 1; you build the SAP side; joint testing on .37 with SAP QA, then shadow mode (decide and log only) for 4 weeks, then live division by division (Developer Brief stages 2.1–2.2).

10. Working rules

- Keep the code in a company Git repository; one branch per change; every release is a versioned zip with release notes (`apply-update.ps1` / `rollback-update.ps1` install and roll back without touching data).
- Do not change decision rules without written approval from the business owner. All rule changes go through `build.py` patches or the server engine, with before/after test results on real data.
- Do not test on .35. Use .37 or your PC.
- Before any release: run the end-to-end tests in `tests/` (they need Python + Playwright) and a role-by-role check (admin, planner, buyer, category head).